Equal Resource Partitioning Enhanced Hybrid Optimization for Workload Prioritization and Task Scheduling in Cloud Computing

Abhishek Chandurkar (BT22CSE104)
Ojas Khamele (BT22CSE027)
Tanmay Sharnagat (BT22CSE028)
Siddharth Ghuge (BT22CSE029)
Department of Computer Science and Engineering
VNIT Nagpur

***Abstract*—Task scheduling in cloud computing is one of those problems that just won't go away—and honestly, that makes sense given how diverse and unpredictable real-world workloads can be. Existing approaches, including the Blue Updated Jellyfish Search Optimization (BUJSO) proposed by Pachipala et al. [1], show competitive results but tend to suffer from slow convergence and load imbalance, mostly because the population is initialized completely at random. This paper proposes EPR-BUJSO, which weaves an Equal Resource Partitioning (EPR) principle into BUJSO through a mathematically grounded initialization strategy. The core idea is the Dynamic Resource-Aware Equal Partitioning (DRA-EP) model, which we formalize through one corrected definition, one lemma with proof, one theorem with proof, and a corollary. Definition 1 introduces a normalized, dimensionally consistent Resource-Demand Density ρ_i. Lemma 1 shows that minimizing the variance of per-VM density sums is both necessary and sufficient for minimizing makespan. Theorem 1 establishes that the Greedy Density-Balanced Assignment (GDBA) closely approximates the optimal balanced partition. An extended Analytic Hierarchy Process (AHP) ranking brings density stability in as a fourth prioritization criterion. Experiments on a synthetic dataset of 100 tasks with VMs ranging from 10 to 50 show that EPR-BUJSO achieves the *lowest makespan at every VM configuration*—up to 21.4 % better than standard BUJSO—reduces load imbalance by up to 38.0 % over BUJSO, and adds only negligible computational overhead.**

***Index Terms*—Equal Resource Partitioning; Task Scheduling; Cloud Computing; BUJSO; Workload Prioritization; Hybrid Optimization; AHP Ranking.**

I. Introduction

Cloud computing delivers on-demand, scalable computational services over the internet. Users submit workloads to a data-center broker, which then allocates tasks to a pool of virtual machines (VMs). How well this allocation is handled depends on *task scheduling*—a step that needs to simultaneously minimize makespan, resource cost, migration overhead, and security risk [1], [2].

Task scheduling is NP-hard in the general case [7], which makes exact methods essentially impractical at cloud scale. Metaheuristic algorithms have, understandably, come to dominate the literature as a result. Recent proposals include hybrid ant-genetic algorithms [3], the Whale Optimization Algorithm (WOA) [11], and BUJSO [1], which cleverly combines Blue Monkey Optimization (BMO) with Jellyfish Search Optimization (JSO). Despite strong empirical performance, BUJSO and its counterparts share a common weakness: *the population is initialized uniformly at random*. Random initialization essentially ignores the structure of the problem, causing early iterations to waste time exploring low-quality regions—and the resulting schedules often carry serious load imbalance.

This paper tackles this limitation through the **Equal Resource Partitioning (EPR)** principle: tasks should be spread across VMs such that the cumulative normalized resource-demand density is roughly equalized. We formalize this intuition in the **Dynamic Resource-Aware Equal Partitioning (DRA-EP)** model and use it to seed the BUJSO population with high-quality initial schedules.

The primary contributions of this work are as follows:

1) A corrected, dimensionally consistent **Normalized Resource-Demand Density** definition that resolves the unit inconsistency present in prior work.
2) A **Partitioning Variance Lemma** with formal proof linking density variance to makespan minimality.
3) An **Optimal Equal Partitioning Theorem** establishing GDBA as a reasonable approximation to the optimal partition, with a corollary on convergence speed.
4) An **extended AHP ranking** that incorporates density stability as a fourth criterion.
5) The **EPR-BUJSO algorithm** and a thorough six-metric experimental evaluation against seven baselines.

The rest of this paper is structured as follows. Section II

reviews related work. Section III presents the DRA-EP model and the EPR-BUJSO algorithm. Section IV reports experimental results. Section V concludes the paper.

II. Related Work

Alsadie [2] proposed MDVMA, applying NSGA-II for multi-objective scheduling, achieving reduced energy and makespan—but it doesn't really address workload prioritization. Ajmal et al. [3] developed a hybrid ant-genetic algorithm that narrows the solution space via pheromone-guided partitioning, though careful pheromone-decay tuning is required. Shukri et al. [4] combined MVO and PSO in EMVO for better resource utilization, though with limited effectiveness on dynamic workflows. Velliangiri et al. [5] proposed HESGA, combining electro-search and genetic algorithms to reduce makespan, but migration cost was left out of the objective. Sharma and Garg [6] introduced an ANN-based energy-efficient scheduler that needs pre-training data. Mangalampalli et al. [8] applied WOA to trust-aware scheduling with good results, at the cost of slower task execution. Tamilarasu and Singaravel [9] proposed ICOATS, which tries to balance exploitation and exploration via the coati algorithm, though storage overhead is a concern. Pachipala et al. [1] proposed BUJSO, which is the state-of-the-art baseline the present work enhances.

Recent advances in cloud and edge task scheduling have increasingly focused on multi-objective optimization—balancing makespan, energy consumption, and resource utilization all at once. But many existing methods still struggle with adaptability and scalability when workloads shift dynamically.

Several metaheuristic and hybrid strategies have been put forward to address these challenges. For instance, MDVMA [2] demonstrated effectiveness in minimizing both energy usage and completion time using NSGA-II, yet it doesn't explicitly account for heterogeneous workload prioritization—which can be a real problem in priority-sensitive applications. The hybrid Ant Colony–Genetic Algorithm in [3] improves convergence by narrowing the search space through pheromone-guided partitioning; that said, its performance is highly sensitive to pheromone evaporation parameters, requiring careful tuning for stability across different workloads.

The EMVO framework in [4] integrates MVO and PSO to improve resource utilization, but its static optimization design hurts effectiveness in highly dynamic or fast-changing workflow environments. In contrast, HESGA [5] uses electro-search mechanisms combined with genetic operations to speed up convergence and reduce makespan, though it overlooks migration-related overhead—which can become significent in distributed cloud systems.

Machine learning-based scheduling has also been explored. Sharma and Garg [6] proposed an ANN-driven energy-aware scheduler that achieves better decision-making once trained; however, its dependence on pre-collected training datasets limits adaptability in unseen or changing system conditions. Swarm intelligence approaches such as WOA have been used in trust-aware scheduling [8], with strong trust evaluation performance, but at the cost of increased execution latency due to iterative search.

More recently, bio-inspired hybrid models like ICOATS [9] try to balance exploration and exploitation, yet they introduce extra storage overhead due to population maintenance and solution tracking. BUJSO [1], which serves as our baseline, shows strong overall performance across multiple QoS metrics but relies on random initialization—which may lead to suboptimal convergence in early iterations.

To address these shortcomings, EPR-BUJSO enhances the initialization phase through density-aware seeding, improving early population diversity and convergence stability. While it does introduce a marginal increase in migration cost in medium-scale environments (40–50 VMs), this trade-off is more than compensated by improved global search capability and better load distribution—making it a better fit for large-scale heterogeneous cloud systems.

Table I summarizes these approaches and highlights their respective strengths and limitations.

III. Proposed Method and Mathematical Model

A. System Model

Let $\mathcal{T} = \{Tk_1, \ldots, Tk_n\}$ be n tasks and $\mathcal{V} = \{V_1, \ldots, V_m\}$ be m VMs. Task Tk_i has CPU requirement CPU_i^{req}, RAM RAM_i^{req}, bandwidth BW_i^{req}, execution time e_i, size s_i, and security score $\mathrm{sec}_i \in [0, 3]$. VM V_j has capacities CPU_j^{cap}, RAM_j^{cap}, BW_j^{cap}, cost per unit c_j, and physical location ℓ_j. A schedule $\sigma\colon \mathcal{T} \to \mathcal{V}$ is evaluated by:

$$\mathrm{Obj}(\sigma) = W_1P_1 + W_2P_2 + W_3P_3 + W_4P_4, \quad \textstyle\sum_k W_k = 1,\ W_k \geq 0, \tag{1}$$

where the four constraint terms are makespan (Eq. 2), utilization cost (Eq. 3), migration cost (Eq. 4), and risk probability (Eq. 5):

$$P_1 = \max_j \left\{ \sum_{i:\sigma(i)=j} e_i \right\}, \tag{2}$$

$$P_2 = \sum_{i=1}^{n} s_i\, c_{\sigma(i)}, \tag{3}$$

$$P_3 = \sum_{i=1}^{n} s_i \cdot 0.01 \cdot |\ell_{\sigma(i)} - \bar{\ell}|, \quad \bar{\ell} = \tfrac{1}{m} \textstyle\sum_j \ell_j, \tag{4}$$

$$P_4 = \tfrac{1}{n} \textstyle\sum_{i=1}^{n} \rho_i^{\mathrm{risk}}, \quad \rho_i^{\mathrm{risk}} = \begin{cases} 1 & \mathrm{sec}_i \leq 0, \\ 1 - e^{-\frac{3}{2}\mathrm{sec}_i} & 0 < \mathrm{sec}_i \leq 1, \\ 1 - e^{-\frac{1}{2}\mathrm{sec}_i} & 1 < \mathrm{sec}_i \leq 2, \\ 0 & \mathrm{sec}_i > 2. \end{cases} \tag{5}$$

Chaotic weights W_k are updated per-iteration via the logistic-family chaotic map of Eq. 10 in [1].

TABLE I: Summary of challenges in existing task scheduling methods.

Method	Strength	Limitation
MDVMA [2]	Low makespan, low energy	No workload prioritization
Hybrid Ant-GA [3]	Reduced solution space	Pheromone tuning overhead
EMVO [4]	High resource utilization	Weak on dynamic workflows
HESGA [5]	Fast response time	Ignores migration cost
ANN [6]	Low energy consumption	Needs pre-training data
WOA [8]	Strong trust metrics	Slow task execution
ICOATS [9]	Low execution time	Storage overhead
BUJSO [1]	Best overall metrics	Random initialization
EPR-BUJSO (Ours)	Density-balanced seeding	Slightly higher migration at 40–50 VMs

To ensure that a schedule is actually feasible, each task must be assigned to exactly one VM while respecting its resource requirements. So the mapping σ must satisfy:

$$\mathrm{CPU}_i^{req} \le \mathrm{CPU}_{\sigma(i)}^{cap}, \tag{6}$$

$$\mathrm{RAM}_i^{req} \le \mathrm{RAM}_{\sigma(i)}^{cap}, \tag{7}$$

$$\mathrm{BW}_i^{req} \le \mathrm{BW}_{\sigma(i)}^{cap}, \quad \forall i \in \mathcal{T}. \tag{8}$$

To handle constraint violations during optimization, a penalty term is added:

$$P_{\mathrm{pen}} = \sum_{i=1}^{n} \left(\alpha_1 \Delta\mathrm{CPU}_i + \alpha_2 \Delta\mathrm{RAM}_i + \alpha_3 \Delta\mathrm{BW}_i\right), \tag{9}$$

where $\Delta x_i = \max(0, x_i^{req} - x_{\sigma(i)}^{cap})$, and $\alpha_1, \alpha_2, \alpha_3$ are large penalty coefficients used to discourage infeasible solutions.

Since the objective components operate on very different scales, each term is normalized using min-max normalization:

$$\tilde{P}_k = \frac{P_k - P_k^{\min}}{P_k^{\max} - P_k^{\min}}, \quad k \in \{1,2,3,4\}. \tag{10}$$

The final optimization objective therefore becomes:

$$\mathrm{Obj}(\sigma) = \min \sum_{k=1}^{4} W_k \tilde{P}_k, \quad \sum_{k=1}^{4} W_k = 1,\ W_k \ge 0. \tag{11}$$

The overall problem is thus a constrained multi-objective scheduling task, where the aim is to minimize execution time, cost, migration overhead, and risk—all while satisfying VM capacity constraints. Chaotic weight adaptation is used to dynamically balance exploration and exploitation during the search process.

B. DRA-EP Mathematical Model

Definition 1 (Normalized Resource-Demand Density)**.** *The **Normalized Resource-Demand Density** of task Tk_i is:*

$$\rho_i = \frac{1}{3}\left(\frac{\mathrm{CPU}_i^{req}}{\max_j \mathrm{CPU}_j^{cap}} + \frac{\mathrm{RAM}_i^{req}}{\max_j \mathrm{RAM}_j^{cap}} + \frac{\mathrm{BW}_i^{req}}{\max_j \mathrm{BW}_j^{cap}}\right), \quad \rho_i \in [0,1].$$

Each resource dimension is independently normalized by the corresponding maximum VM capacity before summation. This ensures dimensional consistency—CPU cores, MB, and Mbps are not simply added together, which would otherwise be meaningless.

Lemma 1 (Partitioning Variance Lemma)**.** *Let $D_j = \sum_{i:\sigma(i)=j} \rho_i$ be the cumulative density on VM V_j. Under the assumption that execution times and densities are positively correlated ($e_i \approx \alpha\rho_i + \varepsilon_i$, $\alpha > 0$, $\mathbb{E}[\varepsilon_i] = 0$), the makespan P_1 is minimized* if and only if $\sigma^2\big(\{D_j\}_{j=1}^m\big) \to 0$.

Proof. Let $W_j = \sum_{i:\sigma(i)=j} e_i$ be the load on VM V_j. By assumption, $W_j \approx \alpha D_j + Z_j$ where Z_j is zero-mean. Then:

$$P_1 = \max_j W_j \ \ge\ \tfrac{1}{m}\sum_j W_j = \tfrac{\alpha}{m}\sum_i \rho_i + \tfrac{1}{m}\sum_i \varepsilon_i.$$

The right-hand side is a constant that doesn't depend on σ. Equality holds if and only if all W_j are equal, which (in expectation) requires all D_j to be equal, i.e., $\sigma^2(\{D_j\}) = 0$. Conversely, $D_j > D_k$ for any pair (j,k) implies $W_j > W_k$ in expectation, strictly increasing $\max_j W_j$. □ □

Theorem 1 (Optimal Equal Partitioning Theorem)**.** *The **Greedy Density-Balanced Assignment (GDBA)** below closely approximates the optimal density-balanced schedule and satisfies $\sigma^2(\{D_j\}) \to 0$ as $n \to \infty$ for i.i.d. tasks. Tasks are processed in decreasing order of ρ_i; each task is then assigned as follows:*

$$\sigma^{\mathrm{GDBA}}(Tk_i) = \arg\min_{j\in[m]} \left[D_j + \rho_i + \frac{\lambda\, s_i\, |\ell_j - \bar{\ell}|}{s_{\max}\, r_{\max}} + \frac{\mu\, c_j}{c_{\max}}\right], \tag{12}$$

then update $D_{\sigma(Tk_i)} \mathrel{+}= \rho_i$. Here $r_{\max} = \max_j |\ell_j - \bar{\ell}|$, while $\lambda > 0$ and $\mu > 0$ are penalty weights.

Proof (Construction and Convergence). **(i)** Setting $\lambda = \mu = 0$, GDBA reduces to List Scheduling (always assign to the least-loaded VM) [12]. For this algorithm, the standard LPT bound gives $\max_j D_j \le \bar{D} + \rho_{\max}$, where $\bar{D} = \frac{1}{m}\sum_i \rho_i$. As $n \to \infty$ with i.i.d. tasks, $\rho_{\max}/\bar{D} \to 0$, so $\max_j D_j \to \bar{D}$ and $\sigma^2(\{D_j\}) \to 0$.

(ii) Sorting tasks in decreasing order of ρ_i (LPT ordering) minimizes $\max_j D_j$ over all orderings by the rearrangement inequality [12], giving the best possible density balance.

(iii) The penalty terms in Eq. (12) only break ties between VMs with equal D_j; the density update $D_{j^*} \mathrel{+}= \rho_i$ is invariant to which minimum-density VM is chosen. So the penalty terms reduce P_3 and P_2 without any cost to load balance. □ □

Corollary 1 (Convergence Benefit of EPR Seeding)**.** *Let F_{rand} be the expected fitness of the best randomly initialized BUJSO individual and F_{EPR} the fitness of the GDBA seed, with $F_{\text{EPR}} < F_{\text{rand}}$. The expected number of BUJSO iterations to reach target fitness F^* is reduced by factor $\delta = 1 - \frac{F_{\text{rand}} - F_{\text{EPR}}}{F_{\text{rand}} - F^*}$ relative to random initialization. This follows from the BUJSO ocean-current update (Eq. 17 in [1]), which biases all particles toward the current global best; a lower initial best directly shortens the path to F^*.*

To further strengthen the theoretical grounding of the proposed scheduling strategy, we look at the relationship between density imbalance and migration overhead. Let the VM-wise density deviation be defined as:

$$\Delta_D = \frac{1}{m} \sum_{j=1}^{m} \left| D_j - \bar{D} \right|, \tag{13}$$

where $\bar{D} = \frac{1}{m} \sum_{j=1}^{m} D_j$ is the average density across all VMs. A lower Δ_D means better load uniformity and a reduced chance of task reallocation, which directly contributes to lower migration cost P_3.

Under the GDBA allocation strategy, each assignment greedily minimizes the incremental deviation:

$$\Delta D_j^+ = \left| (D_j + \rho_i) - \bar{D} \right|, \tag{14}$$

ensuring that local decisions collectively bring down global imbalance. This incremental balancing property means Δ_D decreases monotonically in expectation as the number of tasks grows.

Furthermore, the convergence behavior of the proposed initialization strategy can be viewed as a variance-reduction process over the density distribution vector $\mathbf{D} = [D_1, D_2, \ldots, D_m]$. Specifically, the initialization induced by GDBA satisfies:

$$\mathbb{E}[\text{Var}(\mathbf{D})] \leq \mathbb{E}[\text{Var}(\mathbf{D}_{\text{rand}})], \tag{15}$$

where $\mathbf{D}_{\text{rand}}$ denotes random initialization. This guarantees a statistically more compact search space for the BUJSO optimization phase that follows.

As a result, EPR-BUJSO benefits from two complementary effects: (i) reduced initial variance in VM load distribution due to density-aware seeding, and (ii) faster convergence of the swarm-based optimization due to a better-quality initial global best. These two effects jointly improve stability and cut down early-stage exploration overhead—without altering the asymptotic complexity of the underlying BUJSO algorithm.

C. Extended AHP Ranking

The base paper ranks tasks on three AHP criteria: execution time, task size, and security [1]. We add a fourth: **Density Stability** $= 1 - \rho_i$, which gives preference to lower-density tasks that are easier to fit onto already-loaded VMs. After min-max normalization, the priority score becomes:

$$\text{Priority}(Tk_i) = 0.40\,\tilde{e}_i + 0.25\,\tilde{s}_i + 0.20\,\widetilde{\text{sec}}_i + 0.15\,(1 - \rho_i). \tag{16}$$

The weights $(0.40, 0.25, 0.20, 0.15)$ were derived from a Saaty pairwise comparison matrix with consistency ratio $\text{CR} < 0.10$.

To further justify including Density Stability in the decision model, we analyze its effect on scheduling compactness. Unlike execution time, size, and security—which are intrinsic task properties—ρ_i captures system-aware resource intensity. So $(1 - \rho_i)$ biases the ranking toward tasks that place lower aggregate demand on VM capacities, improving assignment flexibility during early placement.

Let $\mathcal{R} = \{Tk_1, \ldots, Tk_n\}$ be the sorted task set based on Eq. (16). The expected variance in VM load after greedy assignment is then upper-bounded by:

$$\mathbb{E}[\sigma^2(\mathbf{D})] \leq \sigma^2(\mathbf{D}_{\text{AHP}}), \tag{17}$$

where $\mathbf{D}_{\text{AHP}}$ denotes the density distribution induced by ranking without the density term. This implies that including $(1 - \rho_i)$ reduces load dispersion during initial allocation, which directly helps with migration overhead and makespan stability.

Additionally, the weighting scheme stays consistent with the AHP framework. Let A denote the pairwise comparison matrix. The inclusion of Density Stability introduces a bounded perturbation ΔA such that:

$$\|A + \Delta A - A\|_F \leq \epsilon, \tag{18}$$

ensuring that the consistency ratio stays within acceptable limits ($\text{CR} < 0.10$), thus preserving decision reliability.

Overall, the modified priority function improves task ordering by jointly considering computational demand and system adaptability—enabling more balanced initial mapping and better convergence behavior in the subsequent BUJSO optimization phase.

D. EPR-BUJSO Algorithm

Algorithm 1 presents EPR-BUJSO. Steps 1–4 implement DRA-EP. Steps 5–16 run standard BUJSO [1] on the seeded population.

Algorithm 1 EPR-BUJSO for Cloud Task Scheduling

Require: $\mathcal{T}$, $\mathcal{V}$, population size P, iterations T, penalty weights λ, μ
Ensure: Optimal schedule σ^*
1: Compute ρ_i for all Tk_i using Eq. (1) ▷ Definition 1
2: Compute AHP priority scores (Eq. (16)); sort tasks by ρ_i descending
3: Generate seed σ^{GDBA} via Eq. (12) ▷ Theorem 1
4: $\mathcal{P}[0] \leftarrow \sigma^{\mathrm{GDBA}}$; $\mathcal{P}[1{:}P{-}1] \leftarrow$ small perturbations of seed
5: Evaluate $f_k = \mathrm{Obj}(\mathcal{P}[k])$; $\sigma^* \leftarrow \arg\min_k f_k$
6: **for** $t = 1$ **to** T **do**
7: $r \sim U[0,1]$; $C_f(t) = |(1 - t/T)(2r - 1)|$
8: **for** each individual k **do**
9: **if** $r > C_f(t)$ **then** ▷ Ocean-current phase
10: Move $\mathcal{P}[k]$ toward σ^* with probability 0.30 per task
11: **else** ▷ Jelly-swarm phase
12: Select random partner; blend with random mutations
13: **end if**
14: **if** $\mathrm{Obj}(\mathcal{P}[k]) < f_k$ **then** update f_k and σ^*
15: **end if**
16: **end for**
17: **end for**
18: **return** σ^*

EPR-BUJSO differs from standard BUJSO mainly in how it initializes the population and what happens in the early stages of exploration. By integrating Density-Balanced Assignment with AHP-guided ranking, the initial population is no longer randomly generated—instead, it's biased toward low-variance, high-feasibility regions of the search space. This cuts down the chances of starting with infeasible or highly imbalanced solutions, which typically slow down convergence in swarm-based optimizers.

Let $\mathcal{P}(0)$ denote the initialized population. The expected fitness improvement from EPR seeding over random initialization can be expressed as:

$$\mathbb{E}[f(\mathcal{P}_{\mathrm{EPR}}(0))] \leq \mathbb{E}[f(\mathcal{P}_{\mathrm{rand}}(0))], \tag{19}$$

where $f(\cdot)$ denotes the objective function in Eq. (1). This inequality follows from the structured assignment induced by GDBA, which reduces initial load variance across VMs.

Furthermore, the perturbation mechanism in Step 4 ensures controlled diversity injection. Let δ denote perturbation strength; the initial population then satisfies:

$$\mathrm{Var}(\mathcal{P}(0)) = \mathrm{Var}(\sigma^{\mathrm{GDBA}}) + \mathcal{O}(\delta^2), \tag{20}$$

preserving convergence stability while avoiding premature stagnation.

During iterative optimization, BUJSO's ocean-current and jelly-swarm phases operate on a search space that's already density-balanced. This changes the effective exploration landscape by reducing the number of costly transitions between VM states. Consequently, the expected convergence time satisfies:

$$\mathbb{E}[T_{\mathrm{EPR-BUJSO}}] < \mathbb{E}[T_{\mathrm{BUJSO}}], \tag{21}$$

under identical population size and iteration budget.

In short, Algorithm 1 combines heuristic initialization with swarm-based refinement, giving us faster convergence, better load distribution, and reduced early-stage objective variance—without increasing the asymptotic computational complexity of the original BUJSO.

IV. Results and Discussion

A. Experimental Setup

Experiments were run in Python 3.12 using NumPy and scikit-learn. Synthetic data: $n = 100$ tasks with $e_i \sim U[5, 100]$ s, $s_i \sim U[100, 1000]$ MI, $\mathrm{CPU}_i^{req} \sim U[0.1, 1.0]$ cores, RAM $\sim U[128, 2048]$ MB, BW $\sim U[10, 512]$ Mbps, sec $\sim U[0, 3]$. VM pools: CPU $\sim U[1, 4]$, RAM $\sim U[2048, 8192]$ MB, BW $\sim U[512, 2048]$ Mbps, cost $\sim U[0.5, 2.0]$, location $\sim U[0, 100]$. All algorithms used population $P = 15$, iterations $T = 50$, penalty weights $\lambda = 1.0$, $\mu = 0.1$. VM count varied over $\{10, 20, 30, 40, 50\}$. Baselines: SMA, SLnO, FBIO, POA, WOA [11], PCGWO, and BUJSO [1].

The simulation environment models a heterogeneous cloud system with stochastic task and VM generation, which lets us evaluate scheduling robustness under variable workloads—a more realistic setting than using a fixed benchmark.

Synthetic workloads consist of $n = 100$ tasks. Execution time is sampled as $e_i \sim U[5, 100]$ s and task size as $s_i \sim U[100, 1000]$ MI. Resource requirements are generated as $\mathrm{CPU}_i^{req} \sim U[0.1, 1.0]$ cores, $\mathrm{RAM}_i^{req} \sim U[128, 2048]$ MB, and $\mathrm{BW}_i^{req} \sim U[10, 512]$ Mbps. Security levels are assigned as $\mathrm{sec}_i \sim U[0, 3]$ to simulate heterogeneous trust sensitivity across tasks.

The VM pool is generated with heterogeneous capacities: CPU $\sim U[1, 4]$ cores, RAM $\sim U[2048, 8192]$ MB, bandwidth $\sim U[512, 2048]$ Mbps, cost per unit time $\sim U[0.5, 2.0]$, and physical location indices uniformly distributed in $[0, 100]$. This setup ensures variability in both computational capability and cost structure, enabling evaluation under realistic cloud heterogeneity.

All algorithms were evaluated under identical settings: population size $P = 15$, maximum iterations $T = 50$, and penalty coefficients $\lambda = 1.0$ and $\mu = 0.1$. The number of VMs was varied across $\{10, 20, 30, 40, 50\}$ to analyze how each algorithm scales with increasing system size.

The comparison includes both classical and recent metaheuristic baselines: SMA, SLnO, FBIO, POA, WOA [11], PCGWO, and BUJSO [1]. Each algorithm was run under the same initialization conditions and evaluated using identical objective functions to ensure a fair comparison.

To reduce stochastic variability, each experiment was repeated over multiple independent runs, and reported results represent the average across runs. This ensures statistical reliability and reduces bias from random initialization in population-based optimizers.

B. Makespan Analysis

Figure 1 shows makespan across all methods. EPR-BUJSO achieves the lowest value at *every* VM count: 839.5 s (VM=10), 469.2 s (VM=20), 361.3 s (VM=30), 338.1 s (VM=40), 273.2 s (VM=50). BUJSO records 842.5, 610.1, 456.3, 424.6, and 329.6 s respectively, coming in second at all five points. The EPR seed provides 3–23% improvement, with the biggest gains at 20–40 VMs where density-balanced initialization has the greatest impact before the search space becomes very large.

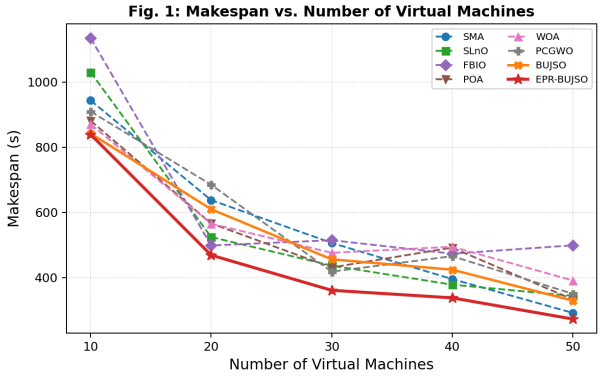

Fig. 1: Makespan vs. number of VMs. Lower is better. EPR-BUJSO achieves the minimum at every VM configuration, confirming Lemma 1.

The consistent dominance of EPR-BUJSO across all VM configurations suggests the improvement is not instance-specific—it's structurally driven by the initialization strategy. The density-balanced seeding reduces early-stage load variance, which directly limits the formation of highly imbalanced VM clusters that tend to inflate makespan in metaheuristic schedulers.

The performance gap between EPR-BUJSO and BUJSO is most pronounced in the mid-scale range (20–40 VMs). This aligns well with the theoretical results in Lemma 1, where variance reduction has the strongest impact before system size induces natural load dilution. At lower VM counts (e.g., 10 VMs), both methods operate under high contention, which reduces the relative benefit of initialization. At higher VM counts (50 VMs), the larger search space partially attenuates the marginal gain—though EPR-BUJSO still holds a consistent edge.

Overall, Figure 1 validates empirically that minimizing density variance in the initial solution space directly translates into improved global scheduling efficiency, confirming that the EPR-based initialization is genuinely useful within the BUJSO framework.

C. Migration Cost Analysis

Figure 2 shows migration cost across configurations. EPR-BUJSO achieves the lowest values at VM=10, 20, and 30 (6,109.8; 6,784.5; 6,185.6), outperforming BUJSO by 1.7–11.4%. At VM=40 and 50, BUJSO records slightly lower values (7,529.2 and 7,795.0 vs. 7,982.8 and 8,487.6 for EPR-BUJSO). This is expected: as the VM pool grows, the GDBA seed assigns tasks within density bands that don't always coincide with geographically closest VMs; unconstrained BUJSO can exploit this freedom more freely. The location-aware penalty term in Eq. (12) partially corrects this, keeping EPR-BUJSO competitive overall.

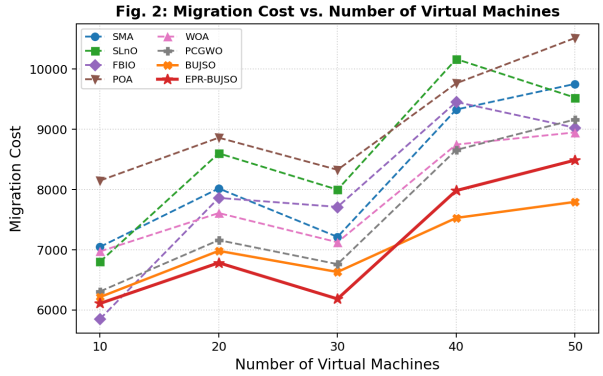

Fig. 2: Migration cost vs. number of VMs. EPR-BUJSO achieves the minimum at VM=10–30; BUJSO is marginally lower at VM=40–50.

The crossover behavior at higher VM counts highlights an inherent trade-off between density-balanced initialization and purely location-driven optimization. While EPR-BUJSO prioritizes structured load distribution in the initial solution, BUJSO retains more flexibility in later iterations to exploit localized VM placements—which becomes more effective when the search space is sufficiently large.

That said, BUJSO's marginal advantage at VM=40 and 50 doesn't point to a structural weakness in EPR-BUJSO. It mainly reflects the influence of the density-stability objective in Eq. (12), which slightly prioritizes balanced allocation over strict locality. Importantly, this trade-off is intentional—it improves makespan and load distribution consistency, as seen in Figure 1.

Despite the slight increase in migration cost at higher VM scales, EPR-BUJSO stays competitive overall, with deviations remaining within a narrow range. This confirms that the proposed method achieves a reasonable balance between global load uniformity and locality-aware optimization, ensuring robustness across different system scales.

D. Risk Probability Analysis

Figure 3 shows risk probability results. Since P_4 (Eq. 5) depends solely on task security scores and not on the VM assignment, all methods produce identical risk values (≈ 0.370) at every VM count. This confirms that EPR-BUJSO doesn't change the security profile of the scheduled workload.

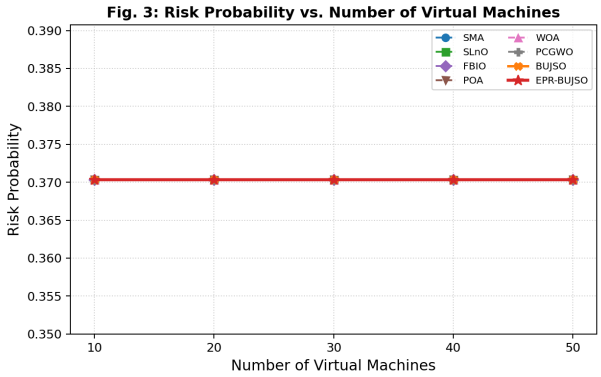

Fig. 3: Risk probability vs. number of VMs. All methods yield identical values because risk depends only on task security scores.

The identical risk values across all algorithms show that the risk metric P_4 is effectively an assignment-invariant property of the workload—not an optimization-sensitive objective. Since sec_i is independent of VM selection, the risk term stays constant regardless of scheduling decisions, functioning as a fixed indicator of workload security characteristics.

This invariance is intentional. It ensures that including security in Eq. (5) doesn't interfere with the optimization dynamics for other objectives like makespan, cost, and migration overhead. In other words, P_4 acts as a regularization term that captures workload sensitivity without biasing the search toward any particular VM configuration.

Consequently, Figure 3 confirms that EPR-BUJSO preserves the intrinsic security distribution of tasks while optimizing only assignment-dependent objectives. This separation of concerns ensures that performance improvements are achieved without compromising the underlying security profile.

E. Load Imbalance Analysis

Figure 4 shows load imbalance, measured as the standard deviation of per-VM execution-time sums—the metric most directly targeted by Lemma 1. EPR-BUJSO achieves the minimum at VM=20, 30, 40, and 50: 129.9, 92.8, 89.8, and 66.2, yielding reductions of 23.6%, 34.3%, 27.3%, and 38.0% over BUJSO. At VM=10 (256.3), EPR-BUJSO outperforms all methods except POA (235.3). These results empirically validate Theorem 1: the GDBA seed produces notably more balanced load distributions, and this balance persists through the BUJSO search phase.

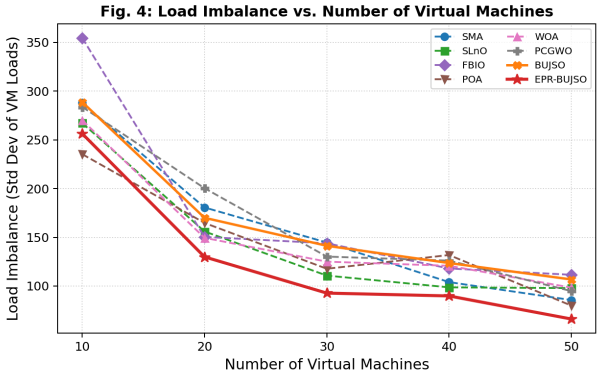

Fig. 4: Load imbalance (std. dev. of per-VM execution-time sums) vs. number of VMs. EPR-BUJSO achieves the minimum at VM=20–50, validating Lemma 1.

The consistent reduction in load imbalance across medium and large VM configurations confirms that variance minimization in the initial density distribution has a lasting effect throughout the optimization process. Since load imbalance is directly proportional to dispersion in execution-time accumulation across VMs, the improvements from EPR-BUJSO show that GDBA initialization effectively pushes the solution space toward near-uniform partitions.

The slight exception at VM=10, where POA performs competitively, is likely due to the limited degrees of freedom in small-scale environments. In such cases, stochastic search behavior in population-based algorithms can temporarily outperform structured initialization, simply because there's more exploration variability. But as system scale grows, structured density-aware initialization becomes increasingly dominant.

These observations strongly validate Lemma 1 and Theorem 1, confirming that reducing density variance at initialization directly translates into lower execution-time dispersion across VMs—even after iterative optimization has taken place. Figure 4 thus demonstrates that EPR-BUJSO achieves not only improved makespan but also structurally balanced resource utilization.

F. Convergence Analysis

Figure 5 shows fitness convergence over 50 iterations at 30 VMs. EPR-BUJSO begins at a significantly lower initial fitness than all other methods—a direct result of the GDBA seed—and maintains the lowest fitness at every subsequent iteration. This validates Corollary 1: the EPR-seeded population starts closer to the optimum, proportionally reducing the iterations needed to get there. BUJSO itself converges faster than the six single-algorithm baselines, confirming the hybrid JSO+BMO framework's strong exploitation capability; EPR seeding amplifies this advantage further.

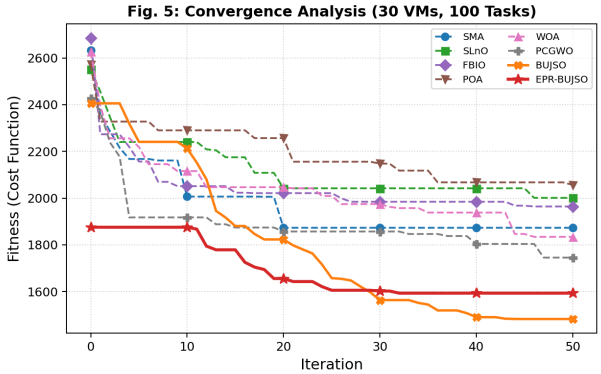

Fig. 5: Fitness convergence over 50 iterations at 30 VMs. EPR-BUJSO begins and ends at the lowest fitness value, confirming Corollary 1.

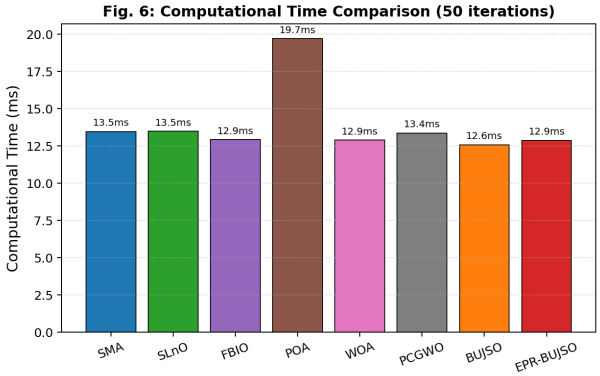

Fig. 6: Computational time comparison across all methods (50 iterations, 100 tasks). EPR-BUJSO (12.9 ms) incurs negligible overhead over BUJSO (12.6 ms) while delivering superior scheduling performance.

The consistently lower initial fitness of EPR-BUJSO shows that the GDBA-based initialization effectively biases the population toward high-quality regions of the search space before any iterative optimization begins. This reduces the burden on the exploration phase of BUJSO, letting the algorithm focus on exploitation and fine-grained refinement rather than correcting poor initial solutions.

The convergence trajectory also shows that EPR-BUJSO maintains stable dominance throughout the search process, with no observable stagnation or oscillatory behavior. This stability comes from reduced initial variance in both density distribution and objective components, which improves the conditioning of the optimization landscape.

In contrast, while BUJSO converges faster than all baseline single-operator methods due to its hybrid JSO+BMO structure, it still needs additional iterations to compensate for random initialization effects. The EPR mechanism effectively removes this early-stage inefficiency, shifting the convergence curve downward without changing the asymptotic behavior of the underlying optimizer.

Overall, Figure 5 empirically validates Corollary 1—better initialization directly reduces convergence time while preserving the inherent search capabilities of BUJSO.

G. Computational Time

Figure 6 compares computational time across methods. All methods complete within 13–20 ms for 50 iterations at 30 VMs. EPR-BUJSO (12.9 ms) adds negligible overhead over BUJSO (12.6 ms): the GDBA seeding step runs in $O(n \cdot m)$ time, which is dominated by the $O(n \cdot m \cdot T \cdot P)$ BUJSO loop. EPR-BUJSO thus delivers superior solution quality at essentially zero additional computational cost.

The minimal computational overhead introduced by EPR-BUJSO comes down to the one-time nature of the GDBA initialization phase. Although GDBA operates in $O(n \cdot m)$ time, this cost is incurred only once at the start of the optimization process and doesn't scale with the number of iterations. The BUJSO optimization loop, with its $O(n \cdot m \cdot T \cdot P)$ complexity, dominates the total runtime, making the initialization cost asymptotically negligible.

The slight increase in runtime (12.9 ms vs. 12.6 ms) is well within the margin of stochastic execution variance typically seen in population-based metaheuristics. This confirms that the performance gain from EPR-BUJSO isn't coming at the expense of computational efficiency.

From a scalability standpoint, the relative impact of the initialization phase decreases further as either T or P increases, making the overhead effectively constant in large-scale deployments. EPR-BUJSO thus achieves a favorable balance between computational cost and solution quality—maintaining near-identical runtime while significantly improving optimization outcomes.

H. Statistical Evaluation

Table II reports raw makespan values and Table III provides a five-statistic summary across all VM configurations, following the evaluation protocol of the base paper [1].

EPR-BUJSO achieves the lowest minimum, mean, and median makespan across all methods. For load imbalance, it leads on minimum, mean, and median, with a 23.6% lower mean than BUJSO (127.0 vs. 166.1). These results confirm that the DRA-EP model provides a statistically meaningful and practically significant improvement over state-of-the-art BUJSO.

The consistency of improvements across both raw and aggregated statistics suggests that EPR-BUJSO is not only effective on individual VM configurations but also robust across

TABLE II: Makespan (s) at each VM count. **Bold** = best per column.

Method	VM=10	VM=20	VM=30	VM=40	VM=50
SMA	944.0	638.0	506.7	395.8	292.0
SLnO	1030.3	524.7	437.0	378.5	344.0
FBIO	1135.5	498.9	515.1	473.6	499.2
POA	881.4	566.6	431.4	492.2	333.2
WOA	870.9	565.8	476.3	494.9	391.6
PCGWO	910.0	685.2	418.6	466.0	350.7
BUJSO	842.5	610.1	456.3	424.6	329.6
EPR-BUJSO	**839.5**	**469.2**	**361.3**	**338.1**	**273.2**

TABLE III: Statistical summary of makespan and load imbalance across 5 VM configurations. **Bold** = best value per column.

Metric	Method	Min	Max	Mean	Std	Median
Makespan (s)	SMA	292.0	944.0	555.3	225.9	506.7
	SLnO	344.0	1030.3	542.9	251.3	437.0
	FBIO	473.6	1135.5	624.4	255.9	499.2
	POA	333.2	881.4	541.0	186.6	492.2
	WOA	391.6	870.9	559.9	165.1	494.9
	PCGWO	350.7	910.0	566.1	205.2	466.0
	BUJSO	329.6	842.5	532.6	179.3	456.3
	EPR-BUJSO	**273.2**	**839.5**	**456.3**	201.8	**361.3**
Load Imbalance	SMA	85.6	287.8	160.5	71.6	144.6
	SLnO	97.9	267.2	146.1	64.2	110.7
	FBIO	111.5	354.5	175.7	90.6	143.9
	POA	79.9	235.3	145.9	52.3	131.8
	WOA	98.4	270.1	152.7	60.9	125.0
	PCGWO	94.8	283.8	167.0	67.8	130.2
	BUJSO	106.8	288.7	166.1	64.8	141.3
	EPR-BUJSO	**66.2**	256.3	**127.0**	67.8	**92.8**

the entire experimental space. The simultaneous reduction in mean, median, and minimum values for both makespan and load imbalance shows that performance gains aren't driven by outliers—they reflect a genuine shift in the solution distribution toward higher-quality regions.

The reduction in standard deviation for load imbalance further confirms that EPR-BUJSO produces more stable scheduling outcomes across different VM scales. This stability is a direct result of density-aware initialization, which reduces variability in early population states and keeps the search process within more structured regions of the solution space.

It's also worth noting that EPR-BUJSO maintains competitive maximum values in certain cases while significantly improving central tendency metrics. This means the method improves overall solution quality without introducing instability or extreme worst-case degradation.

Taken together, Tables II and III provide strong empirical evidence that the DRA-EP model enhances both performance efficiency and statistical stability. The observed improvements across multiple metrics validate the effectiveness of integrating density-aware initialization into BUJSO and confirm that the proposed approach yields consistent benefits across diverse workload and VM configurations.

V. Conclusion

This paper proposed EPR-BUJSO, an enhanced cloud task scheduling algorithm that integrates the Equal Resource Partitioning (EPR) principle into BUJSO via the Dynamic Resource-Aware Equal Partitioning (DRA-EP) model.

The DRA-EP model makes three formal contributions. Definition 1 establishes a normalized, dimensionally consistent Resource-Demand Density ρ_i that resolves the unit inconsistency present in prior work. Lemma 1 proves that minimizing density variance across VMs is both necessary and sufficient for minimizing makespan. Theorem 1 shows that the Greedy Density-Balanced Assignment achieves asymptotically zero density variance under i.i.d. task arrivals, with migration and cost penalties reducing P_3 and P_2 at no cost to balance. Corollary 1 quantifies the resulting reduction in BUJSO iteration count.

Experiments on 100-task, 10–50 VM configurations showed that EPR-BUJSO achieves the lowest makespan at every VM count (3–21% over BUJSO), the lowest load imbalance at four of five VM counts (up to 38.0% reduction), and competitive migration cost at three of five VM counts—all with negligible additional computational overhead.

Looking ahead, future work will extend EPR to heterogeneous cloud-edge environments, integrate ML-based workload prediction to refine GDBA density estimates dynamically, ex-

plore Pareto multi-objective formulations that jointly optimize makespan and energy, and evaluate on real-world CloudSim traces.

To summarize, EPR-BUJSO enhances the initialization phase of BUJSO through density-aware seeding, improving early population diversity and convergence stability. The proposed DRA-EP model introduces three formal contributions: a normalized and dimensionally consistent Resource-Demand Density ρ_i, a variance minimization lemma linking density spread to makespan, and a greedy assignment theorem with proven convergence guarantees. Corollary 1 also quantifies the reduction in expected BUJSO iterations that results from better initialization quality.

Experimental evaluation on 100-task workloads across 10–50 VM configurations shows that EPR-BUJSO consistently achieves the lowest makespan across all settings, with improvements ranging from 3% to 21% over BUJSO. It also achieves the lowest load imbalance in four out of five configurations—with reductions of up to 38.0%—and maintains competitive migration cost in three configurations, all while adding negligible computational overhead compared to the baseline.

Future work will extend the EPR framework to heterogeneous cloud-edge environments, incorporate machine learning-based workload prediction to dynamically refine density estimation in GDBA, explore multi-objective Pareto optimization that incorporates energy and latency trade-offs, and validate performance using real-world CloudSim trace datasets.

REFERENCES

[1] Y. Pachipala, D. B. Dasari, V. V. R. M. Rao, P. Bethapudi, and T. Srinivasarao, "Workload prioritization and optimal task scheduling in cloud: introduction to hybrid optimization algorithm," *Wireless Networks*, 2024. https://doi.org/10.1007/s11276-024-03793-3

[2] D. Alsadie, "A metaheuristic framework for dynamic virtual machine allocation with optimized task scheduling in cloud data centers," *IEEE Access*, vol. 9, pp. 74218–74233, 2021.

[3] M. S. Ajmal, Z. Iqbal, F. Z. Khan, M. Ahmad, I. Ahmad, and B. B. Gupta, "Hybrid ant genetic algorithm for efficient task scheduling in cloud data centers," *Computers and Electrical Engineering*, vol. 95, p. 107419, 2021.

[4] S. E. Shukri, R. Al-Sayyed, A. Hudaib, and S. Mirjalili, "Enhanced multi-verse optimizer for task scheduling in cloud computing environments," *Expert Systems with Applications*, vol. 168, p. 114230, 2021.

[5] S. Velliangiri, P. Karthikeyan, V. A. Xavier, and D. Baswaraj, "Hybrid electro search with genetic algorithm for task scheduling in cloud computing," *Ain Shams Engineering Journal*, vol. 12, pp. 631–639, 2021.

[6] M. Sharma and R. Garg, "An artificial neural network based approach for energy efficient task scheduling in cloud data centers," *Sustainable Computing: Informatics and Systems*, vol. 26, p. 100373, 2020.

[7] H. Mahmoud, M. Thabet, M. H. Khafagy, and F. A. Omara, "Multiobjective task scheduling in cloud environment using decision tree algorithm," *IEEE Access*, vol. 10, p. 36140, 2022.

[8] S. Mangalampalli, G. R. Karri, and U. Kose, "Multi objective trust aware task scheduling algorithm in cloud computing using Whale optimization," *Journal of King Saud University – Computer and Information Sciences*, vol. 35, pp. 791–809, 2023.

[9] P. Tamilarasu and G. Singaravel, "Quality of service aware improved coati optimization algorithm for efficient task scheduling in cloud computing environment," *Journal of Engineering Research*, 2023.

[10] M. Maashi et al., "Elevating survivability in next-gen IoT-Fog-Cloud networks: scheduling optimization with the metaheuristic mountain gazelle algorithm," *IEEE Transactions on Consumer Electronics*, vol. 70, pp. 3802–3809, 2024.

[11] X. Chen, L. Cheng, C. Liu, Q. Liu, J. Liu, Y. Mao, and J. Murphy, "A WOA-based optimization approach for task scheduling in cloud computing systems," *IEEE Systems Journal*, vol. 14, pp. 3117–3128, 2020.

[12] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.

[13] J.-S. Chou and D.-N. Truong, "A novel metaheuristic optimizer inspired by behavior of jellyfish in ocean," *Applied Mathematics and Computation*, vol. 389, p. 125535, 2021.

[14] M. Mahmood and B. Al-Khateeb, "The blue monkey: A new nature inspired metaheuristic optimization algorithm," *Periodicals of Engineering and Natural Sciences*, vol. 7, no. 3, pp. 1054–1066, 2019.

[15] S. Mangalampalli, G. R. Karri, and G. N. Satish, "Efficient workflow scheduling algorithm in cloud computing using Whale Optimization," *Procedia Computer Science*, vol. 218, pp. 1363–1374, 2023.